Data Structures Fundamentals Lab

Data Structures & Algorithms

Lab 02 - Memory, Recursion, Complexity & Dynamic Arrays

Duration: 2 hours (4 challenges x 30 minutes each)

Lab Structure

Each challenge follows this format:

1. **Challenge Presentation** (2 min)
2. **Your Implementation Time** (20 min)
3. **Pseudocode Review** (3 min)
4. **Solution Walkthrough** (5 min)
5. **Code Explanation** (5 min)

Total per challenge: ~35 minutes

Today's Topics

Memory Management in C

- Stack vs. Heap allocation
- Dynamic memory with `malloc()` and `free()`

Recursion vs. Iteration

- Solving the same problem both ways
- Understanding call stacks

Algorithm Complexity

- Counting operations
- Comparing approaches

Dynamic Arrays

- Arrays that grow when needed
- Amortized analysis in practice

Challenge 1

Memory Detective: Track the Leaks

Challenge 1: Problem Statement

Task: You are a "memory detective" investigating a suspicious program. Build a simple **memory tracker** that logs every allocation and deallocation, and reports any memory leaks at the end.

Requirements:

- Create a tracker that stores addresses of allocated memory blocks
- Implement `tracked_malloc(size)` - allocates memory and logs it
- Implement `tracked_free(ptr)` - frees memory and removes log entry
- Implement `report_leaks()` - prints any unfreed allocations
- Use a fixed-size array (max 100 allocations) to store tracked pointers

Time: 20 minutes

Challenge 1: Pseudocode

```
ALGORITHM MemoryTracker

  1. Define tracker structure:
     - array of 100 pointers (void*)
     - count of active allocations

  2. tracked_malloc(size):
     - Call malloc(size) to get pointer
     - If pointer is not NULL:
       * Store pointer in tracker array
       * Increment count
       * Print "ALLOC: address [count active]"
     - Return pointer

  3. tracked_free(ptr):
     - Search tracker array for ptr
     - If found: remove from array, decrement count
     - Call free(ptr)
     - Print "FREE: address [count active]"

  4. report_leaks():
     - If count > 0: Print each unfreed address
     - If count == 0: Print "No leaks detected!"
END ALGORITHM
```

Challenge 1: Solution Code

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_TRACKED 100

// Memory tracker structure
struct MemoryTracker {
    void* allocations[MAX_TRACKED];
    int count;
};

// Global tracker
struct MemoryTracker tracker = { .count = 0 };
```

Challenge 1: Solution Code (continued)

```c
// Tracked allocation
void* tracked_malloc(size_t size) {
    void* ptr = malloc(size);

    if (ptr != NULL && tracker.count < MAX_TRACKED) {
        tracker.allocations[tracker.count] = ptr;
        tracker.count++;
        printf("  ALLOC: %p [%d active]\n", ptr, tracker.count);
    }

    return ptr;
}
```

Challenge 1: Solution Code (continued)

```c
// Tracked deallocation
void tracked_free(void* ptr) {
    if (ptr == NULL) return;

    // Find and remove from tracker
    for (int i = 0; i < tracker.count; i++) {
        if (tracker.allocations[i] == ptr) {
            // Shift remaining entries left
            for (int j = i; j < tracker.count - 1; j++) {
                tracker.allocations[j] = tracker.allocations[j + 1];
            }
            tracker.count--;
            printf("  FREE:  %p [%d active]\n", ptr, tracker.count);
            break;
        }
    }

    free(ptr);
}
```

Challenge 1: Solution Code (Leak Report)

```c
// Report memory leaks
void report_leaks() {
    printf("\n--- Memory Leak Report ---\n");

    if (tracker.count == 0) {
        printf("  No leaks detected! Clean program.\n");
    } else {
        printf("  WARNING: %d leak(s) found!\n", tracker.count);
        for (int i = 0; i < tracker.count; i++) {
            printf("  LEAKED: %p\n", tracker.allocations[i]);
        }
    }

    printf("--------------------------\n");
}
```

Challenge 1: Solution Code (Test Cases)

```c
int main() {
    printf("=== Scenario 1: Good programmer ===\n");
    int* a = (int*)tracked_malloc(sizeof(int));
    int* b = (int*)tracked_malloc(5 * sizeof(int));
    tracked_free(a);
    tracked_free(b);
    report_leaks();
    // Output: No leaks detected!

    printf("\n=== Scenario 2: Careless programmer ===\n");
    int* x = (int*)tracked_malloc(sizeof(int));
    int* y = (int*)tracked_malloc(sizeof(int));
    int* z = (int*)tracked_malloc(sizeof(int));
    tracked_free(x);
    // Forgot to free y and z!
    report_leaks();
    // Output: WARNING: 2 leak(s) found!

    return 0;
}
```

Challenge 1: Code Explanation

Part 1: The Tracker Structure

```
struct MemoryTracker {
    void* allocations[MAX_TRACKED];  // Array of generic pointers
    int count;                        // How many active allocations
};
```

Why `void*`?

- `void*` is a generic pointer that can point to any data type
- We don't care *what* is allocated, just *where* it is
- Works for `int*`, `char*`, `float*`, any pointer type

Why a fixed array?

- Simple approach, no dynamic allocation for the tracker itself
- Avoids the awkward situation of tracking the tracker's own memory

Challenge 1: Code Explanation

Part 2: Tracked Allocation

```
void* tracked_malloc(size_t size) {
    void* ptr = malloc(size);
    if (ptr != NULL && tracker.count < MAX_TRACKED) {
        tracker.allocations[tracker.count] = ptr;
        tracker.count++;
    }
    return ptr;
}
```

Flow:

```
tracked_malloc(sizeof(int))
  1. malloc(4) returns address 0x7f8b00
  2. Store 0x7f8b00 in tracker.allocations[0]
  3. tracker.count becomes 1
  4. Return 0x7f8b00 to caller
```

This is a **wrapper function** - it adds behaviour on top of an existing function.

Challenge 1: Code Explanation

Part 3: The Removal Shift

```
for (int j = i; j < tracker.count - 1; j++) {
    tracker.allocations[j] = tracker.allocations[j + 1];
}
```

Why shift elements?

When removing from the middle of an array, we must fill the gap:

```
Before freeing allocations[1]:
  [0x100] [0x200] [0x300]    count = 3
              ^
           remove this

After shifting:
  [0x100] [0x300] [  ?  ]    count = 2
```

This is an O(n) operation! We'll learn about this cost in detail later.

Challenge 1: Code Explanation

Part 4: Why Memory Leaks Are Dangerous

Small program: A few bytes leaked, no real harm

Long-running server:

```
Server runs for 30 days
Leaks 100 bytes every request
10,000 requests per day

Total leaked: 100 x 10,000 x 30 = 30,000,000 bytes = 30 MB
```

Eventually: Out of memory, program crashes!

Real-world disasters:

- Video game servers slowing down over time
- Web browsers becoming sluggish after hours of use
- Medical devices malfunctioning on extended shifts

Challenge 2

Staircase Climber: Recursion vs. Iteration

Challenge 2: Problem Statement

Task: You're climbing a staircase with `n` steps. Each time, you can take **1 step** or **2 steps**. How many **distinct ways** can you reach the top?

Examples:

```
n = 1:  1 way    (1)
n = 2:  2 ways   (1+1, 2)
n = 3:  3 ways   (1+1+1, 1+2, 2+1)
n = 4:  5 ways   (1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2)
```

Requirements:

- Implement `climbStairs_recursive(n)` using recursion
- Implement `climbStairs_iterative(n)` using a loop
- Add a **step counter** to each function to count operations
- Compare the number of steps for n = 10, 20, 30

Time: 20 minutes

Challenge 2: Pseudocode

```
ALGORITHM ClimbStairs_Recursive(n, counter)
  1. Increment counter by 1

  2. Base cases:
     - If n == 1: return 1
     - If n == 2: return 2

  3. Recursive case:
     - return ClimbStairs_Recursive(n-1) + ClimbStairs_Recursive(n-2)
END ALGORITHM

ALGORITHM ClimbStairs_Iterative(n, counter)
  1. If n <= 2: return n

  2. prev1 = 1, prev2 = 2

  3. For i = 3 to n:
     - Increment counter
     - current = prev1 + prev2
     - prev1 = prev2
     - prev2 = current

  4. Return prev2
END ALGORITHM
```

Challenge 2: Solution Code

```c
#include <stdio.h>

// Global counters to track operations
long long recursive_steps = 0;
long long iterative_steps = 0;

// Recursive approach
int climbStairs_recursive(int n) {
    recursive_steps++;  // Count this call

    // Base cases
    if (n == 1) return 1;
    if (n == 2) return 2;

    // Recursive case
    return climbStairs_recursive(n - 1)
         + climbStairs_recursive(n - 2);
}
```

Challenge 2: Solution Code (continued)

```
// Iterative approach
int climbStairs_iterative(int n) {
    if (n <= 2) return n;

    int prev1 = 1;  // Ways to climb 1 step
    int prev2 = 2;  // Ways to climb 2 steps
    int current;

    for (int i = 3; i <= n; i++) {
        iterative_steps++;  // Count this operation
        current = prev1 + prev2;
        prev1 = prev2;
        prev2 = current;
    }

    return prev2;
}
```

Challenge 2: Solution Code (Test Cases)

```c
int main() {
    int test_values[] = {10, 20, 30, 40};

    printf("%-6s %-12s %-18s %-18s\n",
           "n", "Ways", "Recursive Steps", "Iterative Steps");
    printf("-------------------------------------------------------\n");

    for (int t = 0; t < 4; t++) {
        int n = test_values[t];

        // Reset counters
        recursive_steps = 0;
        iterative_steps = 0;

        int result_iter = climbStairs_iterative(n);
        int result_rec = (n <= 30) ? climbStairs_recursive(n) : -1;

        printf("%-6d %-12d %-18lld %-18lld\n",
               n, result_iter, recursive_steps, iterative_steps);
    }

    return 0;
}
```

Challenge 2: Code Explanation

Part 1: Why This Problem Is Fibonacci

Insight: The staircase problem IS the Fibonacci sequence!

```
Steps: 1   2   3   4   5   6   7    8
Ways:  1   2   3   5   8   13  21   34
         ↑ Fibonacci numbers!
```

Why? To reach step n:

- You came from step n-1 (took 1 step), or
- You came from step n-2 (took 2 steps)
- Total ways = ways(n-1) + ways(n-2) -- This IS Fibonacci!

Challenge 2: Code Explanation

Part 2: Recursive Call Tree for n=5

```
                    climb(5)
                   /        \
            climb(4)      climb(3)
           /       \      /      \
      climb(3)  climb(2) climb(2) climb(1)
      /     \      |        |        |
  climb(2) climb(1) 2        2        1
     |        |
     2        1
```

Notice: `climb(3)` is calculated **twice**, `climb(2)` is calculated **three** times!

This is massively wasteful. Function calls grow exponentially.

Challenge 2: Code Explanation

Part 3: Iterative Approach - Rolling Window

```
int prev1 = 1;  // Ways for step 1
int prev2 = 2;  // Ways for step 2

for (int i = 3; i <= n; i++) {
    current = prev1 + prev2;  // Just add previous two
    prev1 = prev2;            // Slide window forward
    prev2 = current;
}
```

Trace for n = 6:

Step i	prev1	prev2	current
3	1	2	3
4	2	3	5
5	3	5	8

Challenge 2: Code Explanation

Part 4: The Shocking Comparison

n	Recursive Steps	Iterative Steps	Speedup
10	109	8	14x
20	13,529	18	752x
30	1,664,079	28	59,431x
40	204,668,309	38	5,386,008x

Key takeaway:

- Recursive: **O(2^n)** - exponential growth!
- Iterative: **O(n)** - linear growth!

At n = 40: Recursion does 204 million operations vs. 38 iterations.

This is why algorithm analysis matters!

Challenge 2: Code Explanation

Part 5: Memory Usage Comparison

Recursive: Each call uses stack space

```
climb(5) → climb(4) → climb(3) → climb(2) → returns
                                              ↑
                        Maximum stack depth = n-1 frames
```

Stack usage: **O(n)**

Iterative: Only 3 variables regardless of n

```
prev1, prev2, current → always just 3 integers → 12 bytes
```

Stack usage: **O(1)**

For very large n: Recursion can cause a **stack overflow** -- the program crashes because the call stack runs out of space!

Challenge 3

Race the Algorithms: Complexity Counter

Challenge 3: Problem Statement

Task: Build an **Algorithm Racing Arena** that runs different search algorithms on the same data and counts exactly how many comparisons each one makes.

Requirements:

- Generate an array of `n` sorted integers
- Implement `linear_search(arr, n, target)` with a comparison counter
- Implement `binary_search(arr, n, target)` with a comparison counter
- Run both on the same array and target
- Display a race-style comparison table for n = 100, 1000, 10000, 100000

Bonus: Search for a target that does NOT exist (worst case!)

Time: 20 minutes

Challenge 3: Pseudocode

```
ALGORITHM SearchRace

  1. generate_sorted_array(arr, n):
     - Fill arr with values: arr[i] = i * 2

  2. linear_search(arr, n, target, comparisons):
     - For i = 0 to n-1:
       * Increment comparisons
       * If arr[i] == target: return i
     - Return -1 (not found)

  3. binary_search(arr, n, target, comparisons):
     - low = 0, high = n - 1
     - While low <= high:
       * mid = (low + high) / 2
       * Increment comparisons
       * If arr[mid] == target: return mid
       * If arr[mid] < target: low = mid + 1
       * Else: high = mid - 1
     - Return -1 (not found)

  4. For each n in {100, 1000, 10000, 100000}:
     - Generate array, run both searches
     - Print comparison counts
END ALGORITHM
```

Challenge 3: Solution Code

```c
#include <stdio.h>
#include <stdlib.h>

// Generate sorted array: [0, 2, 4, 6, 8, ...]
void generate_sorted_array(int* arr, int n) {
    for (int i = 0; i < n; i++) {
        arr[i] = i * 2;
    }
}

// Linear search with comparison counter
int linear_search(int* arr, int n, int target, int* comparisons) {
    *comparisons = 0;

    for (int i = 0; i < n; i++) {
        (*comparisons)++;
        if (arr[i] == target) {
            return i;
        }
    }

    return -1;  // Not found
}
```

Challenge 3: Solution Code (continued)

```c
// Binary search with comparison counter
int binary_search(int* arr, int n, int target, int* comparisons) {
    *comparisons = 0;
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;  // Avoids overflow!
        (*comparisons)++;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return -1;  // Not found
}
```

Challenge 3: Solution Code (Race Function)

```c
void run_race(int n) {
    int* arr = (int*)malloc(n * sizeof(int));
    generate_sorted_array(arr, n);

    // Search for a value that does NOT exist (worst case)
    int target = n * 2 + 1;

    int linear_comps, binary_comps;

    linear_search(arr, n, target, &linear_comps);
    binary_search(arr, n, target, &binary_comps);

    // Calculate speedup
    double speedup = (double)linear_comps / binary_comps;

    printf("| %-10d | %-12d | %-12d | %-10.1fx |\n",
           n, linear_comps, binary_comps, speedup);

    free(arr);
}
```

Challenge 3: Solution Code (Test Cases)

```c
int main() {
    printf("=== Algorithm Racing Arena ===\n");
    printf("Target: does NOT exist (worst case)\n\n");
    printf("| %-10s | %-12s | %-12s | %-10s |\n",
           "Array Size", "Linear", "Binary", "Speedup");
    printf("|------------|--------------|--------------|------------|\n");

    run_race(100);
    run_race(1000);
    run_race(10000);
    run_race(100000);

    printf("\n");

    return 0;
}

/* Expected Output:
| Array Size | Linear       | Binary       | Speedup    |
|------------|--------------|--------------|------------|
| 100        | 100          | 7            | 14.3x      |
| 1000       | 1000         | 10           | 100.0x     |
| 10000      | 10000        | 14           | 714.3x     |
| 100000     | 100000       | 17           | 5882.4x    |
*/
```

Challenge 3: Code Explanation

Part 1: The Comparison Counter Trick

```
int linear_search(int* arr, int n, int target, int* comparisons) {
    *comparisons = 0;
    // ...
    (*comparisons)++;
```

Why pass `int* comparisons`?

- We need the counter to survive after the function returns
- Passing a **pointer** lets the function modify the caller's variable
- This is C's way of "returning" extra information

Why `(*comparisons)++` and not `*comparisons++`?

- `*comparisons++` would increment the **pointer** first (wrong!)
- `(*comparisons)++` increments the **value** the pointer points to (correct!)
- Operator precedence: `++` binds tighter than `*`

Challenge 3: Code Explanation

Part 2: Binary Search - Avoiding Overflow

```
int mid = low + (high - low) / 2;
```

Why not just `(low + high) / 2`?

For very large arrays:

```
low  = 2,000,000,000
high = 2,100,000,000

low + high = 4,100,000,000   (overflows 32-bit int!)

But:
high - low     = 100,000,000
(high-low) / 2 = 50,000,000
low + 50,000,000 = 2,050,000,000   (works!)
```

This is a classic bug that was in Java's standard library for 9 years before being discovered!

Challenge 3: Code Explanation

Part 3: Understanding the Results

Linear Search (O(n)):

```
n = 100      → 100 comparisons
n = 1000     → 1000 comparisons        (10x more data = 10x more work)
n = 10000    → 10000 comparisons       (100x more data = 100x more work)
n = 100000   → 100000 comparisons      (1000x more data = 1000x more work)
```

Linear growth: work grows directly with data size

Binary Search (O(log n)):

```
n = 100      → 7 comparisons           log2(100) ≈ 6.6
n = 1000     → 10 comparisons          log2(1000) ≈ 10
n = 10000    → 14 comparisons          log2(10000) ≈ 13.3
n = 100000   → 17 comparisons          log2(100000) ≈ 16.6
```

Logarithmic growth: 10x more data = only 3-4 more comparisons!

Challenge 3: Code Explanation

Part 4: Why Binary Search Requires Sorted Data

Binary search ONLY works on sorted arrays. Why?

The algorithm assumes: "If middle element is too big, discard right half"

```
Sorted:     [2, 5, 8, 12, 15, 20, 25]
Search: 15
Middle: 12 → 15 > 12, search right half  [CORRECT]

Unsorted:   [8, 2, 25, 5, 15, 12, 20]
Search: 15
Middle: 5  → 15 > 5, search right half → [12, 20]
             15 is NOT in right half!     [WRONG!]
```

The trade-off:

- Sorting costs O(n log n) upfront
- But if you search many times, the investment pays off
- Search K times: O(n log n + K log n) vs O(K * n)

Challenge 4

Build a Stretchy Array (Dynamic Array)

Challenge 4: Problem Statement

Task: Build a **Dynamic Array** (also called a "stretchy array") that automatically grows when full. This is how Python lists and Java ArrayList work internally!

Requirements:

- Create a `DynamicArray` struct with `data`, `size`, and `capacity`
- Implement `da_create(initial_capacity)` - create new dynamic array
- Implement `da_push(array, value)` - add element, resize if needed
- Implement `da_get(array, index)` - access element by index
- Implement `da_print(array)` - display contents and stats
- Implement `da_destroy(array)` - free all memory
- Track and display the number of resize operations

Time: 20 minutes

Challenge 4: Pseudocode

```
ALGORITHM DynamicArray

  1. da_create(initial_capacity):
     - Allocate DynamicArray struct
     - Allocate data array of initial_capacity
     - Set size = 0, capacity = initial_capacity, resizes = 0
     - Return pointer

  2. da_push(array, value):
     - If size == capacity:
       * Double the capacity
       * Allocate new, larger array
       * Copy all elements
       * Free old array
       * Increment resize counter
     - array.data[size] = value
     - Increment size

  3. da_get(array, index):
     - If index < 0 or index >= size: error
     - Return array.data[index]

  4. da_destroy(array):
     - Free data array
     - Free struct
END ALGORITHM
```

Challenge 4: Solution Code

```c
#include <stdio.h>
#include <stdlib.h>

// Dynamic Array structure
typedef struct {
    int* data;        // Pointer to underlying array
    int size;         // Number of elements currently stored
    int capacity;     // Total slots available
    int resizes;      // Number of times we resized
} DynamicArray;

// Create a new dynamic array
DynamicArray* da_create(int initial_capacity) {
    DynamicArray* da = (DynamicArray*)malloc(sizeof(DynamicArray));
    da->data = (int*)malloc(initial_capacity * sizeof(int));
    da->size = 0;
    da->capacity = initial_capacity;
    da->resizes = 0;
    return da;
}
```

Challenge 4: Solution Code (continued)

```c
// Push element to end (resize if needed)
void da_push(DynamicArray* da, int value) {
    // Check if we need to grow
    if (da->size == da->capacity) {
        int new_capacity = da->capacity * 2;

        printf("  RESIZE: %d -> %d (copying %d elements)\n",
               da->capacity, new_capacity, da->size);

        // Allocate bigger array
        int* new_data = (int*)malloc(new_capacity * sizeof(int));

        // Copy existing elements
        for (int i = 0; i < da->size; i++) {
            new_data[i] = da->data[i];
        }

        // Free old array, use new one
        free(da->data);
        da->data = new_data;
        da->capacity = new_capacity;
        da->resizes++;
    }

    // Insert the element
    da->data[da->size] = value;
    da->size++;
}
```

Challenge 4: Solution Code (continued)

```c
// Get element at index (with bounds checking)
int da_get(DynamicArray* da, int index) {
    if (index < 0 || index >= da->size) {
        printf("ERROR: Index %d out of bounds (size: %d)\n",
               index, da->size);
        return -1;
    }
    return da->data[index];
}

// Print array contents and stats
void da_print(DynamicArray* da) {
    printf("  Contents: [");
    for (int i = 0; i < da->size; i++) {
        printf("%d", da->data[i]);
        if (i < da->size - 1) printf(", ");
    }
    printf("]\n");
    printf("  Size: %d  |  Capacity: %d  |  Resizes: %d\n",
           da->size, da->capacity, da->resizes);
}

// Free all memory
void da_destroy(DynamicArray* da) {
    free(da->data);
    free(da);
}
```

Challenge 4: Solution Code (Test Cases)

```c
int main() {
    printf("=== Building a Stretchy Array ===\n\n");

    // Start with capacity of just 2
    DynamicArray* arr = da_create(2);

    printf("Inserting 10 elements into capacity-2 array:\n");
    for (int i = 1; i <= 10; i++) {
        printf("Push %d: ", i);
        da_push(arr, i * 10);
        printf("  [size=%d, cap=%d]\n", arr->size, arr->capacity);
    }

    printf("\nFinal state:\n");
    da_print(arr);

    printf("\nAccess test:\n");
    printf("  arr[0] = %d\n", da_get(arr, 0));
    printf("  arr[4] = %d\n", da_get(arr, 4));
    printf("  arr[9] = %d\n", da_get(arr, 9));
    da_get(arr, 15);  // Intentional out-of-bounds

    da_destroy(arr);
    return 0;
}
```

Challenge 4: Code Explanation

Part 1: The Dynamic Array Structure

```
typedef struct {
    int* data;        // The actual array in heap memory
    int size;         // How many items we have
    int capacity;     // How many items we CAN have
    int resizes;      // How many times we doubled
} DynamicArray;
```

Analogy: A backpack with stretchy pockets

- `size` = items currently in backpack
- `capacity` = current pocket size
- When pocket is full, swap for a bigger backpack

Why `typedef`?

Without: `struct DynamicArray da;`

With: `DynamicArray da;` -- cleaner!

Challenge 4: Code Explanation

Part 2: The Doubling Resize Strategy

```
if (da->size == da->capacity) {
    int new_capacity = da->capacity * 2;  // DOUBLE!
    int* new_data = malloc(new_capacity * sizeof(int));
    for (int i = 0; i < da->size; i++) {
        new_data[i] = da->data[i];  // Copy everything
    }
    free(da->data);       // Free old array
    da->data = new_data; // Use new array
}
```

Why double (2x) instead of adding 1?

Adding 1 each time: resize happens EVERY insert

- 10 inserts: 0+1+2+3+4+5+6+7+8+9 = 45 copies (O(n^2) total!)

Doubling each time: resize happens rarely

- 10 inserts: 1+2+4 = 7 copies (O(n) total!)

Challenge 4: Code Explanation

Part 3: Execution Trace

Starting with capacity = 2:

Push #	Value	Size Before	Capacity	Resize?	Copies
1	10	0	2	No	0
2	20	1	2	No	0
3	30	2	2->4	Yes	2
4	40	3	4	No	0
5	50	4	4->8	Yes	4
6	60	5	8	No	0
7	70	6	8	No	0
8	80	7	8	No	0

Challenge 4: Code Explanation

Part 4: Bounds Checking

```
int da_get(DynamicArray* da, int index) {
    if (index < 0 || index >= da->size) {
        printf("ERROR: Index %d out of bounds\n", index);
        return -1;
    }
    return da->data[index];
}
```

Why is this important?

C does NOT check array bounds automatically!

```
int arr[5] = {10, 20, 30, 40, 50};
printf("%d", arr[10]);  // C allows this! Reads garbage from memory.
```

Our dynamic array is **safer** than a raw C array because we check!

Languages like Java and Python throw exceptions for out-of-bounds access. We're

Challenge 4: Code Explanation

Part 5: Memory Management - Two-Level Freeing

```
void da_destroy(DynamicArray* da) {
    free(da->data);  // First: free the internal array
    free(da);        // Then: free the struct itself
}
```

Why two `free()` calls?

Our structure has **two** heap allocations:

```
Stack:                    Heap:
  da (pointer) ──────> [DynamicArray struct]
                          data ──────> [int array...]
                          size
                          capacity
```

Order matters!

- If we free `da` first, we lose access to `da->data` (memory leak!)

Lab Session Complete!

Summary: What You Built

Challenge 1: Memory Detective

- Tracked allocations and deallocations
- Discovered memory leaks automatically

Challenge 2: Staircase Climber

- Solved same problem with recursion and iteration
- Measured the explosive cost of naive recursion

Challenge 3: Algorithm Racing Arena

- Raced linear search vs. binary search
- Saw O(n) vs O(log n) with real numbers

Challenge 4: Stretchy Array

- Built a dynamic array from scratch
- Experienced amortized O(1) insertion firsthand

Key Concepts Reinforced

Memory Management

- Every `malloc()` needs a matching `free()`
- Memory leaks are silent and cumulative
- Always free inner allocations before outer ones

Recursion vs. Iteration

- Same problem, vastly different performance
- Recursion can be elegant but expensive
- Iteration often wins for simple patterns

Key Concepts Reinforced (continued)

Algorithm Complexity

- Counting operations reveals true performance
- O(log n) is dramatically faster than O(n) at scale
- Theory matches practice -- the numbers don't lie

Dynamic Arrays

- Fixed arrays can grow by doubling capacity
- Amortized O(1): rare expensive operations average out
- This is how real-world list implementations work

Common Mistakes to Avoid

Memory Management:

```
free(da);
free(da->data);  // ERROR: da is already freed!
// Fix: Free inner (da->data) before outer (da)
```

Recursion Without Base Case:

```
int climb(int n) {
    return climb(n-1) + climb(n-2);  // No base case = infinite recursion!
}
```

Binary Search on Unsorted Data:

```
int arr[] = {5, 1, 9, 3, 7};
binary_search(arr, 5, 3, &comps);  // WRONG result!
// Binary search requires sorted array!
```

Common Mistakes to Avoid (continued)

Operator Precedence with Pointers:

```
*comparisons++    // Increments the pointer (wrong!)
(*comparisons)++  // Increments the value (correct!)
```

Forgetting Bounds Check:

```
return da->data[index];  // If index >= size, reads garbage!
// Always check: if (index >= da->size) { error; }
```

Integer Overflow in Binary Search:

```
int mid = (low + high) / 2;          // Can overflow!
int mid = low + (high - low) / 2;    // Safe!
```

Complexity Summary

Challenge	Key Algorithm	Time Complexity
1. Memory Tracker: tracked_free	Linear search + shift	O(n)
2a. Staircase: Recursive	Fibonacci-style recursion	O(2^n)
2b. Staircase: Iterative	Rolling window	O(n)
3a. Linear Search	Check every element	O(n)
3b. Binary Search	Halve search space	O(log n)
4. Dynamic Array: Push	Doubling resize	O(1) amortized